

Studi Empiris *Flaky Test* di GitHub melalui *Mining Commit Message “flaky”*, *Screening Repositori*, dan *Commit Diff Analysis* dengan Validasi *Reexecution* Terbatas

Rega Saputra¹, Najma Khonsa Tsabita², Aulia Febriyanti³, Chalimatus Sa’Diyah⁴, Raden Ngabei Jabbar⁵
Information System, Faculty of Engineering and Computer Science, Bakrie University, Jakarta 12940, Indonesia
Email: 1232002070@student.bakrie.ac.id¹, 1232002090@student.bakrie.ac.id²,
1232002014@student.bakrie.ac.id³, 1232002046@student.bakrie.ac.id⁴,
1232002078@student.bakrie.ac.id⁵

ABSTRAK

Flaky test merupakan tantangan dalam pengembangan perangkat lunak karena menghasilkan *output* pengujian tidak konsisten tanpa ada perubahan fungsional pada kode. Berbagai penelitian telah mengusulkan pendekatan deteksi dan klasifikasi, tapi pembuktian *flaky test* berbasis sinyal empiris dari *commit history* GitHub masih belum banyak dikaji. Penelitian ini mengusulkan *pipeline repository mining* berbasis komit dengan memanfaatkan kata kunci “flaky”, pemeriksaan keberadaan *automated test* dan CI *workflow*, serta analisis perubahan kode melalui *commit diff* sebagai bukti utama keterkejadian dan strategi perbaikan *flakiness*. *Rule-based labeling* berbasis taksonomi literatur dipakai untuk klasifikasi penyebab *flaky*, termasuk *async wait*, *concurrency*, dan *test order dependency*. Hasil penelitian berupa dataset repositori kandidat, instrumen pelabelan penyebab, dan protokol *pipeline* berbasis komit yang dapat direplikasi kembali untuk mendukung riset *flaky test* yang lebih *auditable*, *traceable*, dan minim bias.

Kata Kunci: *repository mining*, *flaky test*, *commit history*, *commit diff*, *rule-based labeling*.

I. PENDAHULUAN

Automated testing tidak terpisahkan dalam pengembangan perangkat lunak, terutama yang menerapkan *continuous integration* (CI). *Test suite* yang stabil membantu tim mendeteksi regresi lebih cepat dan mempercepat siklus rilis [1], [2]. Tapi, dalam praktiknya, tidak semua kegagalan pengujian otomatis berkaitan dengan bug fungsional. Banyak kegagalan muncul secara tak menentu dan sulit direproduksi, yang pada akhirnya menurunkan kepercayaan developer terhadap hasil pengujian [3], [4].

Flakiness pada pengujian otomatis telah diamati dalam berbagai level pengujian, termasuk pengujian berbasis UI dan *web testing*, yang sering melibatkan *event asynchronous*, *shared state*, serta isu waktu (*timing*) yang sulit dikontrol [5], [6], [7]. Sejumlah penelitian menunjukkan bahwa *test flakiness* berdampak langsung pada produktivitas developer,

karena waktu analisis lebih banyak tersita untuk memverifikasi apakah kegagalan tersebut merupakan bug nyata atau hanya ketidakstabilan pengujian [4], [8], [9]. Penelitian industri berskala besar bahkan menemukan bahwa biaya *test reruns* yang berulang di CI dapat menjadi signifikan, baik dari sisi waktu maupun konsumsi komputasi [1].

Berbagai pendekatan telah dikembangkan untuk mendeteksi juga mengklasifikasikan *flaky tests*, mulai dari teknik *rerunning test* [10], *framework* berbasis *machine learning* [11], [12], [13], hingga pendekatan *few-shot learning* untuk memprediksi kategori penyebab *flakiness* [12]. Walau begitu, sebagian besar penelitian fokus pada peningkatan teknik deteksi atau klasifikasi di *level test case*, sementara pendekatan untuk menambang bukti *flaky test* secara empiris langsung dari *commit history* GitHub secara terstruktur masih belum banyak dikaji. Padahal, *commit message* dan *code diff* menyimpan sinyal perbaikan nyata yang sering kali menjadi sumber informasi paling reliabel mengenai kapan *flakiness* muncul dan bagaimana developer meresponsnya [3], [8], [14].

Kesenjangan ini memotivasi penelitian untuk menggabungkan *repository mining* berbasis kata kunci “flaky” dalam *commit message*, pemeriksaan keberadaan *automated test* di repositori, serta analisis perubahan kode melalui *commit diff*. Pendekatan berbasis komit dipandang penting karena merepresentasikan bukti perbaikan yang benar-benar dilakukan developer, bukan hasil observasi manual yang rawan bias atau *noise* [3], [6], [15]. Selain itu, beberapa studi menegaskan bahwa reproduksi *flaky test* di lingkungan lokal memang tidak selalu tinggi, sehingga validasi melalui *reexecution* terbatas tetap bernilai sebagai penguat bukti *nondeterminism*, walau tidak menjadi metode utama [1], [8].

Berdasarkan motivasi tersebut, penelitian ini diarahkan untuk menjawab kebutuhan akan *pipeline* pembuktian *flaky test* di GitHub yang dapat ditelusuri, diverifikasi, dan direplikasi, dengan menempatkan

analisis pada level komit sebagai sumber bukti utama keterkejadian *flaky* dan respons perbaikannya.

II. TINJAUAN PUSTAKA dan KERANGKA KONSEPTUAL

2.1 *Flaky Test*

Merupakan *test* otomatis yang menghasilkan *output* tidak konsisten (*pass/fail*) ketika dijalankan berulang pada versi kode yang sama, tanpa perubahan fungsional pada sistem yang diuji [8]. Karakter nondeterministik ini menjadi pembeda utama antara *flaky test* dan *bug* regresi, di mana kegagalan *flaky* tidak selalu menandakan kesalahan logika program, melainkan ketidakstabilan eksekusi pengujian itu sendiri [2].

Flakiness pada *test suite* terbukti menimbulkan dampak negatif baik di tingkat developer maupun organisasi. Studi industri dan empiris berskala besar menunjukkan bahwa *flaky test* meningkatkan biaya komputasi pada CI akibat *test reruns*, memperpanjang waktu investigasi kegagalan, serta menurunkan kepercayaan developer terhadap hasil *automated test* [1], [8], [15], [16], [17]. Romano et al. juga menegaskan bahwa *flaky test* pada pengujian berbasis UI berkontribusi pada *testing distrust*, yang menghambat proses verifikasi bug karena sulit direproduksi secara konsisten [5]. Selain itu, penelitian industrial oleh Lam et al. menhighlight bahwa *root cause investigation flaky test* dapat mengonsumsi waktu yang signifikan dalam organisasi dengan *test suite* besar [3].

Luo et al. melalui investigasi empiris berskala besar pada proyek open-source Java mengidentifikasi tiga kategori penyebab dominan *flakiness*, ada *Async Wait*, *Concurrency*, dan *Test Order Dependency (TOD)* [8]. *Taxonomy flaky test* juga berkembang ke konteks ekosistem lain. Beberapa studi *machine learning* mengelompokkan penyebab *flaky test* menggunakan model prediktif dan *few-shot learning*, seperti yang dilakukan pada *framework FlaKat* dan *FlakyCat* [1], [12], [13]. Literatur *multivocal review* juga merangkum taksonomi yang lebih luas yang mencakup *network*, *timing*, *I/O*, *randomness*, *floating-point operations*, *unordered collections*, dan faktor lingkungan lainnya, yang dapat bervariasi antar bahasa dan *pipeline* eksekusi [6], [17].

Penelitian ini mengadopsi taksonomi berbasis literatur sebagai instrumen labeling dengan pendekatan *rule-based coding*.

2.2 *Commit Diff Analysis*

Analisis *commit diff* dipakai dalam penelitian empiris untuk mengamati perubahan kode sebelum dan sesudah sebuah *commit*, dengan tujuan memahami

tindakan perbaikan yang dilakukan developer sekaligus mengekstraksi indikasi akar masalah *flakiness*. Pendekatan berbasis komit ini telah digunakan dalam studi skala besar untuk mengidentifikasi *commit* yang berpotensi memperbaiki *flaky tests* melalui inspeksi perubahan kode dan pola perbaikannya [1], [3], [8], [11]. Dengan demikian, analisis *commit diff* tidak digunakan untuk mendefinisikan *flaky*, tapi sebagai teknik pembuktian empiris berbasis perubahan kode nyata yang terdokumentasi.

2.3 *Continuous Integration (CI)*

Merupakan praktik menjalankan proses *build* dan pengujian otomatis setiap kali terjadi perubahan kode. Lingkungan CI berperan sebagai *symptom amplifier* bagi *flakiness*, karena *test* dijalankan secara berulang dalam *environment* yang terstandar, sehingga memperjelas gejala nondeterminisme yang tidak mudah teramati pada eksekusi tunggal. Sejumlah studi industri menegaskan bahwa biaya *flaky tests* pada CI dapat menjadi signifikan akibat *reruns* yang berulang dan investigasi kegagalan palsu [16], [17].

Pada GitHub, *pipeline* CI dapat didefinisikan melalui keberadaan *workflow* otomatis seperti GitHub *Actions*, yang mengeksekusi test dalam lingkungan terstandar dan mendokumentasikan status integrasi melalui *commit status* API atau *workflow run logs* [17].

2.4 *Commit-Based Flaky Test Repository Mining Pipeline*

Penelitian empiris perlu proses pengumpulan bukti yang terstruktur dan dapat diaudit ulang, apalagi untuk membedakan antara *noise commit* dan sinyal perbaikan *flaky tests* yang valid. Karenanya, penelitian ini mengadopsi kerangka konseptual berbasis komit, yang sejalan dengan studi yang memanfaatkan *regression-test history data* dan *flaky test prediction instruments* sebagai dasar pelabelan penyebab [1], [2], [13], [16], [17].

Kerangka konseptual penelitian dirancang sebagai berikut:

Input: Repositori publik GitHub yang memiliki *commit message* mengandung kata *flaky*.

Proses utama:

1. Penambahan *commit* kandidat *flaky test*
2. Deduplikasi repositori unik
3. *Screening* keberadaan *automated test* dan CI
4. *Sampling* repositori/*commit* yang valid
5. Analisis perubahan kode melalui *commit diff*
6. Pelabelan penyebab *flaky* dan strategi perbaikannya berbasis taksonomi literatur (*rule-based labeling*)
7. Validasi *reexecution* terbatas pada subset terpilih sebagai penguat bukti (opsional)

Output: Dataset repositori kandidat, labeling instrument, protokol analisis berbasis komit, serta log

eksekusi proses penelitian yang dapat diperiksa dan direplikasi kembali [2], [13].

III. METODOLOGI

3.1 Desain Penelitian

Penelitian ini menggunakan desain *empirical repository mining* yang digabungkan dengan analisis kualitatif berbasis taksonomi (*qualitative coding*), dan divalidasi melalui *re-execution* testing terbatas. Desain ini dipilih karena penelitian berfokus pada pembuktian empiris fenomena *flaky test* berdasarkan riwayat *commit* developer, bukan pembuatan *tools* baru, dan pemahaman penyebab serta strategi perbaikan *flakiness* berdasarkan perubahan kode yang nyata di GitHub.

Commit message yang mengandung kata “*flaky*” digunakan sebagai *trigger* bahwa developer telah menandai isu pada *automated test*. *Commit* kandidat kemudian dianalisis melalui *commit diff* untuk menilai apakah perubahan memang menyasar *test suite* atau *pipeline* CI, lalu diklasifikasikan menggunakan taksonomi penyebab dan strategi fix dari literatur yang mapan.

3.2 Objek dan Unit Analisis

Objek penelitian ini adalah proyek perangkat lunak *open-source* publik di GitHub yang menerapkan *automated testing*. Dengan populasi konseptual yaitu semua proyek *open-source* di GitHub yang menggunakan *automated test*, tapi penelitian dibatasi pada *repository* yang memiliki *commit message* eksplisit “*flaky*”, agar data relevan dan berbasis pengakuan langsung dari developer.

Untuk unit analisis utamanya adalah *Commit* (hash, pesan, dan perubahan kode). Sementara unit analisis sekunder adalah *Repository*, digunakan hanya untuk menyediakan konteks kelayakan.

3.3 Sumber Data dan Alat

3.3.1 GitHub REST API & GitHub CLI

Dataset awal *commit* diambil secara otomatis melalui GitHub REST API (v3) menggunakan GitHub CLI (gh api) dengan endpoint pencarian *commit* GET */search/commits*.

Developer *commit* yang mengandung kata *flaky* kemudian diekstrak metadata-nya dan disimpan dalam format NDJSON dan CSV untuk proses screening dan analisis lanjutan.

3.3.2 PNPM Workspace untuk Eksekusi Test

Karena banyak *repository* JavaScript berbentuk monorepo, pengujian ulang dilakukan menggunakan PNPM *workspace* agar *script test* dapat dijalankan hanya pada *package* yang memang memiliki *automated test*, jadi proses validasi tidak terganggu oleh *script build/lint/coverage* yang tidak relevan.

3.3.3 Alat Analisis

1. *JSON filtering* untuk ekstraksi metadata API,
2. *Commit diff analysis* untuk melihat perubahan test/CI,
3. *Rule-based labeling* untuk klasifikasi penyebab *flaky* dan strategi fix.

3.4 Prosedur Pengumpulan Data (*Pipeline*)

Table 3.1 Tahap Proses dan *Output*

Tahap	Proses	Output
1	<i>Mining commit</i> message “ <i>flaky</i> ” via API	Metadata <i>commit</i> kandidat (NDJSON/CSV)
2	Deduplikasi <i>repository</i> unik	Daftar <i>repository</i> relevan
3	Screening <i>eligibility</i> (memastikan ada <i>automated test/CI evidence</i>)	<i>Repository eligible</i> + catatan mitigasi
4	<i>Filtering commit</i> hanya dari <i>repository eligible</i>	<i>Dataset final commit</i> untuk dianalisis
5	<i>Sampling commit</i> (random dengan <i>fixed seed</i>)	<i>Subset commit</i> untuk <i>qualitative coding</i>
6	<i>Labeling</i> penyebab + strategi fix berdasarkan taksonomi	Tabel hasil analisis <i>flaky/bug/infra/unclear</i>
7	<i>Re-execution</i> testing terbatas (≥ 3 run per <i>subset</i>)	Bukti <i>PASS/FAIL divergence</i> sebagai validasi pendukung

3.5 Instrumen Analisis dan Pelabelan

3.5.1 Taksonomi Penyebab *Flaky*

Untuk menjaga validitas dan keterbandingan, penelitian menggunakan kategori penyebab *flaky test* dari literatur yang sudah mapan: *Async Wait/Timing*, *Concurrency/Race Condition*, *Test Order Dependency*, *External Dependency*, *Resource Leak*, *Clock/Timezone Issue*, *Randomness/Seed*, *Unordered Collections*, *Platform/Environment Issue*, dan *Unclear/Other*.

3.5.2 Taksonomi Strategi *Fix* Developer

Developer fix diklasifikasikan berdasarkan pola perubahan kode pada *commit*, meliputi: *Explicit wait* menggantikan *sleep*, Sinkronisasi *teardown/queue flush*, Isolasi *shared state*, *Mock/Stubbing dependency*, Penetapan *random seed*, *Retry mechanism* (sebagai mitigasi, bukan bukti akar masalah), dan Stabilisasi CI.

3.5.3 *Dual Coding Subset*

Sebagai validasi *labeling*, dilakukan *dual coding* pada 10–15 *commit* secara independen menggunakan aturan yang sama, lalu dibandingkan menggunakan *percentage agreement*. Perbedaan label dibahas reflektif untuk memperjelas instrumen, tanpa mengubah data utama.

3.6 Validasi Eksperimen: *Re-Execution Testing* Terbatas

Re-execution test dilakukan pada *subset repository* yang memungkinkan dijalankan lokal, minimal N=3 kali per *commit* yang sama, untuk mengamati perilaku nondeterministik *PASS/FAIL*. Peran validasi ini sebagai bukti pendukung bahwa test bersifat flaky (jika hasil berbeda antar run), juga bukan replikasi CI secara penuh, dan bukan sumber klaim utama flakiness, sehingga tetap minim bias dan realistis.

3.7 Alur Bukti Analisis dalam Penelitian

Mining commit message "flaky"

1. Deduplikasi *repository*
2. *Screening eligibility* (memastikan *automated test evidence* itu ada)
3. *Filtering dataset final*
4. *Sampling subset commit*
5. *Commit diff analysis*
6. *Rule-based labeling (Flaky vs Bug vs Infra vs Unclear)*
7. *Re-execution* terbatas sebagai penguat
8. Penyusunan hasil analisis untuk paper.

IV. HASIL DAN PEMBAHASAN

4.1 Hasil Pengujian

Penambahan *commit history* GitHub pada *repository work-squared* menghasilkan temuan bahwa sinyal *flaky test* tidak cukup dibuktikan melalui label *commit message* semata, melainkan harus diperkuat oleh perbedaan hasil pengujian pada *commit* yang sama saat dieksekusi berulang di *pipeline* CI. Dari 3 run pengujian terbatas (Run-1, Run-4, Run-12), diperoleh pola hasil yang mengindikasikan *flakiness*, *bug* fungsional, dan kegagalan infrastruktur sebagai berikut.

4.1.1 *Flakiness* di *Suite Message Queue Cleanup*

Suite Message Queue Cleanup menunjukkan hasil tidak konsisten di *commit* identik:

Table 4.1 Hasil dan Indikasi Utama Test Run

Test Run	Hasil	Indikasi Utama
Run-1	FAIL	<i>Cleanup</i> belum selesai saat <i>assert</i> dijalankan
Run-4	PASS	Kondisi eksekusi ideal, <i>cleanup</i> selesai tepat waktu
Run-12	FAIL	Error identik dengan Run-1

Pola *FAIL*, *PASS*, *FAIL* pada *commit* yang sama ini memenuhi karakteristik *flaky test*, yaitu status pengujian berubah antar run tanpa perubahan kode sistem. Hal ini menunjukkan bahwa *pipeline* mengeksekusi *assertion check* sebelum proses *teardown async* selesai, jadinya hasilnya

dipengaruhi oleh kondisi eksekusi *pipeline* dan bukan oleh bug fungsional aplikasi.

4.1.2 Kegagalan Konsisten di *Suite InputValidator*

Berbeda dari *suite flaky*, *InputValidator* selalu gagal di semua run:

Table 4.2 Hasil dan Temuan Test Run

Test Run	Hasil	Temuan
Run-1	FAIL	6 test case gagal
Run-4	FAIL	6 test case gagal
Run-12	FAIL	6 test case gagal

Kegagalan terjadi secara konsisten dan dapat direproduksi, terutama pada fungsi sanitasi input seperti pembersihan *newline*, penghapusan tag HTML/XML, dan deteksi input berbahaya. Karena tidak ada perbedaan status antar run, kegagalan ini bukan *flaky test*, melainkan *defect fungsional* pada *logic validator*.

4.1.3 Isu *Better-SQLite3 Binding*

Error binding SQLite muncul di CI, tetapi:

1. Hanya memengaruhi test yang memanggil *DB native module*,
2. Tidak menyebabkan seluruh *suite* gagal, dan
3. Tidak berkorelasi dengan perubahan *PASS/FAIL* di *suite Message Queue Cleanup*.

Hal ini menunjukkan bahwa *failure infrastruktur* perlu dibedakan dari klaim *flaky test*, karena merupakan masalah lingkungan eksekusi, bukan nondeterminisme *test logic*.

4.2 Penyelesaian Masalah

Berdasarkan penelusuran *commit message* yang mengandung sinyal *flaky* pada repository GitHub *work-squared*, *developer* menyelesaikan masalah *flaky test* dengan pendekatan yang berorientasi pada stabilitas *pipeline* pengujian dan kejelasan hasil tes.

4.2.1 Perbaikan pada Workflow CI

Developer melakukan penyederhanaan *workflow .github/workflows/test.yml* untuk memastikan test berjalan lebih lengkap dan tidak terlewat. Sebelumnya, CI hanya menjalankan test pada satu *package* tertentu, yang menyebabkan kegagalan test di bagian server tidak pernah terlihat di *pipeline* CI.

Konfigurasi CI sebelum perbaikan:

- name: Run unit tests

run: pnpm --filter @work-squared/web test

Konfigurasi CI setelah perbaikan:

- name: Run all tests in monorepo

run: pnpm test

Perubahan ini membuat *pipeline* CI menjadi lebih reliabel dan mengungkap 7 kegagalan test lama yang sebelumnya tersembunyi akibat test

skipping, sehingga kondisi *flakiness* menjadi lebih *visible* dan dapat ditindaklanjuti.

4.2.2 Penambahan Konfigurasi *Build Environment*

Developer menstabilkan proses *native module build* di CI, yang sebelumnya memicu kegagalan test karena dependensi sistem tidak tersedia. Perbaikan ini dilakukan dengan menambahkan instalasi dependensi sistem dan konfigurasi *.npmrc* untuk mengaktifkan *build script* dependensi *native*.

```
- name: Install system dependencies
```

```
run: |
```

```
    sudo apt-get update
```

```
    sudo apt-get install -y build-essential
```

```
python3
```

```
enable-pre-post-scripts=true
```

```
optional=true
```

Langkah ini memperbaiki *failure* yang bukan berasal dari logic test, tapi dari lingkungan build CI yang tidak mendukung kompilasi modul *native*, yang merupakan salah satu sumber kegagalan nondeterministik dalam pipeline.

4.2.3 Pembersihan Broken Test dan Sensitif Timing

Developer memilih untuk menghapus test yang rusak dan sensitif terhadap timing eksekusi (timing-dependent), yang sering menyebabkan *failure* sporadis, termasuk test yang memicu *unhandled promise rejection*. Fokus pengujian kemudian diarahkan kembali ke skenario fungsional inti yang dapat diuji secara stabil. Seperti salah satu contoh dibawah ini

```
while (!task1CanComplete) {
    await new Promise(resolve =>
        setTimeout(resolve, 10))
}
```

Penghapusan test berbasis loop delay kecil dan skenario queue destruction timing ini menunjukkan bahwa developer mengidentifikasi *flakiness* sebagai masalah desain test, bukan bug sistem, lalu mengeliminasi sumber nondeterminisme tersebut agar test suite kembali dipercaya.

4.2.4 Penghapusan Noise Failure

Developer juga menghapus *step coverage upload* yang memicu kegagalan pipeline karena *folder coverage/* belum dikonfigurasi dan tidak ada di repository saat CI berjalan. Penghapusan ini tidak memperbaiki logic test, tetapi menghilangkan indikator kegagalan palsu yang mengganggu interpretasi pipeline pengujian

Fokus Perbaikan	Penyebab Flakiness	Aksi Fix Developer	Dampak
Eksekusi test CI	Test parsial, <i>silent skipping</i>	Gunakan <i>pnpm test</i> di root	Failure lama menjadi terlihat
Build environment	Error binding modul <i>native</i>	Install dependensi sistem + <i>.npmrc</i>	Test <i>native</i> stabil di CI
Test suite design	Mocking kompleks, sensitif timing	Hapus test <i>flaky/broken</i>	Test suite menjadi deterministik
Pipeline noise	Path coverage mismatch	Hapus step sementara	CI lebih bersih, minim kegagalan palsu

V. KESIMPULAN

Fenomena *flaky test* tidak dapat dibuktikan hanya dari kegagalan pengujian tunggal atau sekadar munculnya kata “*flaky*” pada *commit message*. Penelitian ini menunjukkan bahwa sinyal *flakiness* harus dianalisis secara berlapis, pada bukti nyata dari *commit history*, *code diff*, dan perilaku test saat dieksekusi ulang pada *commit* yang sama.

Dari tiga file yang dianalisis, ditemukan pola yang berbeda:

Message Queue Cleanup terbukti memenuhi karakteristik *flaky test*, karena menunjukkan hasil FAIL → PASS → FAIL pada *commit* identik tanpa perubahan kode.

InputValidator mengalami kegagalan konsisten di seluruh test run, sehingga dikategorikan sebagai bug fungsional pada logic validasi input, bukan *flaky test*.

Error pada modul *better-sqlite3* binding hanya memengaruhi sebagian test yang memanggil *native DB* module dan tidak menyebabkan keluaran nondeterministik antar run, sehingga diklasifikasikan sebagai *infra failure/environment issue*, bukan sumber utama *flakiness*.

Strategi perbaikan yang dilakukan developer juga memperkuat temuan, di mana perbaikan lebih banyak diarahkan pada stabilisasi pipeline CI dan desain *test suite*, bukan pada perubahan fungsional sistem. Langkah seperti menjalankan seluruh *test* monorepo, menyiapkan dependensi sistem di CI, serta menghapus *test broken* yang sensitif *timing* terbukti meningkatkan visibilitas kegagalan lama, determinisme *test suite*, dan kebersihan pipeline, sekaligus mengurangi kegagalan palsu yang mengganggu interpretasi hasil pengujian.

Dengan demikian, penelitian ini berhasil membuktikan bahwa *flaky test* adalah masalah eksekusi test dan *pipeline*, bukan selalu bug sistem, serta menegaskan pentingnya analisis berbasis komit sebagai pendekatan yang *traceable*, dan minim bias dalam menambang bukti *flakiness* di repository open-source GitHub.

Table 4.2 Pola Penyebab dan Penyelesaian Flaky Test pada Pipeline CI Berdasarkan Commit Developer

DAFTAR PUSTAKA

- [1] F. Leinen, D. Elsner, A. Pretschner, A. Stahlbauer, M. Sailer, and E. Jürgens, “Cost of Flaky Tests in Continuous Integration: An Industrial Case Study,” in *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*, Toronto, ON, Canada: IEEE, May 2024, pp. 329–340. doi: 10.1109/ICST60714.2024.00037.
- [2] S. Vysali, S. McIntosh, and B. Adams, “Quantifying, Characterizing, and Mitigating Flakily Covered Program Elements,” *IEEE Trans. Software Eng.*, vol. 48, no. 3, pp. 1018–1029, Mar. 2022, doi: 10.1109/TSE.2020.3010045.
- [3] W. Lam, P. Godefroid, S. Nath, A. Santhiar, and S. Thummalapenta, “Root causing flaky tests in a large-scale industrial setting,” in *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis*, Beijing China: ACM, July 2019, pp. 101–111. doi: 10.1145/3293882.3330570.
- [4] M. Eck, F. Palomba, M. Castelluccio, and A. Bacchelli, “Understanding flaky tests: the developer’s perspective,” in *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, Tallinn Estonia: ACM, Aug. 2019, pp. 830–840. doi: 10.1145/3338906.3338945.
- [5] A. Romano, Z. Song, S. Grandhi, W. Yang, and W. Wang, “An Empirical Analysis of UI-based Flaky Tests,” Mar. 03, 2021, *arXiv*: arXiv:2103.02669. doi: 10.48550/arXiv.2103.02669.
- [6] A. Tahir, S. Rasheed, J. Dietrich, N. Hashemi, and L. Zhang, “Test flakiness’ causes, detection, impact and responses: A multivocal review,” *Journal of Systems and Software*, vol. 206, p. 111837, Dec. 2023, doi: 10.1016/j.jss.2023.111837.
- [7] Y. Pei, J. Sohn, S. Habchi, and M. Papadakis, “Time-based Repair for Asynchronous Wait Flaky Tests in Web Testing,” May 19, 2023, *arXiv*: arXiv:2305.08592. doi: 10.48550/arXiv.2305.08592.
- [8] Q. Luo, F. Hariri, L. Eloussi, and D. Marinov, “An empirical analysis of flaky tests,” in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, Hong Kong China: ACM, Nov. 2014, pp. 643–653. doi: 10.1145/2635868.2635920.
- [9] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, “Surveying the developer experience of flaky tests,” in *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, Pittsburgh Pennsylvania: ACM, May 2022, pp. 253–262. doi: 10.1145/3510457.3513037.
- [10] J. Bell, O. Legunsen, M. Hilton, L. Eloussi, T. Yung, and D. Marinov, “D E F LAKER: automatically detecting flaky tests,” in *Proceedings of the 40th International Conference on Software Engineering*, Gothenburg Sweden: ACM, May 2018, pp. 433–444. doi: 10.1145/3180155.3180164.
- [11] S. Lin, R. Z. H. Liu, and L. Tahvildari, “FlaKat: A Machine Learning-Based Categorization Framework for Flaky Tests,” Mar. 01, 2024, *arXiv*: arXiv:2403.01003. doi: 10.48550/arXiv.2403.01003.
- [12] A. Akli, G. Haben, S. Habchi, M. Papadakis, and Y. Le Traon, “FlakyCat: Predicting Flaky Tests Categories using Few-Shot Learning,” in *2023 IEEE/ACM International Conference on Automation of Software Test (AST)*, Melbourne, Australia: IEEE, May 2023, pp. 140–151. doi: 10.1109/AST58925.2023.00018.
- [13] W. Lam, R. Oei, A. Shi, D. Marinov, and T. Xie, “iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests,” in *2019 12th IEEE Conference on Software Testing, Validation and Verification (ICST)*, Xi’an, China: IEEE, Apr. 2019, pp. 312–322. doi: 10.1109/ICST.2019.00038.
- [14] P. Wendler and S. Winter, “Regression-Test History Data for Flaky-Test Research,” in *Proceedings of the 1st International Workshop on Flaky Tests*, Lisbon Portugal: ACM, Apr. 2024, pp. 3–4. doi: 10.1145/3643656.3643901.
- [15] M. Gruber, S. Lukasczyk, F. Kroiß, and G. Fraser, “An Empirical Study of Flaky Tests in Python,” in *2021 14th IEEE Conference on Software Testing, Verification and Validation (ICST)*, Apr. 2021, pp. 148–158. doi: 10.1109/ICST49551.2021.00026.
- [16] P. Zhang, Y. Jiang, A. Wei, V. Stodden, D. Marinov, and A. Shi, “Domain-Specific Fixes for Flaky Tests with Wrong Assumptions on Underdetermined Specifications,” in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, Madrid, ES: IEEE, May 2021, pp. 50–61. doi: 10.1109/ICSE43902.2021.00018.
- [17] O. Parry, G. M. Kapfhammer, M. Hilton, and P. McMinn, “Evaluating Features for Machine Learning Detection of Order- and Non-Order-Dependent Flaky Tests,” in *2022 IEEE Conference on Software Testing, Verification and Validation (ICST)*, Valencia, Spain: IEEE, Apr. 2022, pp. 93–104. doi: 10.1109/ICST53961.2022.00021.